

```
#comparison
print("Original Token Count :", len(tokens))
print("After Stop Word Removal :", len(filtered_tokens))

print("\nVocabulary Before :", len(set(tokens)))
print("Vocabulary After :", len(set(filtered_tokens)))
```

```
Original Token Count : 127
After Stop Word Removal : 79
```

```
Vocabulary Before : 107
Vocabulary After : 75
```

```
print("Original Tokens\n")
print(tokens)

print("\n-----")

print("\nStop Words Removed\n")
print(filtered_tokens)

print("\n-----")

print("\nStemmed Tokens\n")
print(stemmed_words)

print("\n-----")

print("\nLemmatized Tokens\n")
print(lemmatized_words)
```

```
Original Tokens
```

```
['hey', 'everyone', 'im', 'really', 'excited', 'because', 'ive', 'just', 'completed', 'my', 'first', 'machine', 'learning', 'project', 'on']
-----
```

```
Stop Words Removed
```

```
['hey', 'everyone', 'im', 'really', 'excited', 'ive', 'completed', 'first', 'machine', 'learning', 'project', 'rd', 'july', 'project', 'an']
-----
```

```
Stemmed Tokens
```

```
['hey', 'everyon', 'im', 'realli', 'excit', 'ive', 'complet', 'first', 'machin', 'learn', 'project', 'rd', 'juli', 'project', 'analyz', 'c']
-----
```

```
Lemmatized Tokens
```

```
['hey', 'everyone', 'im', 'really', 'excited', 'ive', 'completed', 'first', 'machine', 'learning', 'project', 'rd', 'july', 'project', 'an']
```

✓ EXP 5

```
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping chunkers/maxent_ne_chunker.zip.
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Unzipping corpora/words.zip.
True
```

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.tag import pos_tag
from nltk.chunk import ne_chunk
```

```
text = ""
```

```
Five Feet Apart is a romantic drama directed by Justin Baldoni and released by CBS Films and Lionsgat
The movie stars Haley Lu Richardson as Stella Grant and Cole Sprouse as Will Newman, and the story ta
The film raises awareness about Cystic Fibrosis and became popular in countries such as India, Canada
Overall, Five Feet Apart is an emotional and inspiring movie with excellent performances, and I would
""
```

```
print(text)
```

```
Five Feet Apart is a romantic drama directed by Justin Baldoni and released by CBS Films and Lionsgate in 2019.
The movie stars Haley Lu Richardson as Stella Grant and Cole Sprouse as Will Newman, and the story takes place in Saint Grace Hospital in 1
The film raises awareness about Cystic Fibrosis and became popular in countries such as India, Canada, the United Kingdom, and Australia t
Overall, Five Feet Apart is an emotional and inspiring movie with excellent performances, and I would rate it 4.5 out of 5.
```

```
import nltk
nltk.download('punkt_tab')
from nltk.tokenize import word_tokenize
tokens = word_tokenize(text)
print(tokens)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
['Five', 'Feet', 'Apart', 'is', 'a', 'romantic', 'drama', 'directed', 'by', 'Justin', 'Baldoni', 'and', 'released', 'by', 'CBS', 'Films',
```

```
import nltk
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('tagsets_json')
from nltk.tag import pos_tag
tagged_tokens = pos_tag(tokens)
print(tagged_tokens)
```

```
nltk.help.upenn_tagset('NNP')
```

```
[('Five', 'CD'), ('Feet', 'NNP'), ('Apart', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('romantic', 'JJ'), ('drama', 'NN'), ('directed', 'VBN'),
NNP: noun, proper, singular
    Motown Venneboenger Czystochwa Ranzer Conchita Trumplane Christos
    Oceanside Escobar Kreisler Sawyer Cougar Yvette Ervin ODI Darryl CTCA
    Shannon A.K.C. Meltex Liverpool ...
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]   date!
[nltk_data] Downloading package tagsets_json to /root/nltk_data...
[nltk_data]   Unzipping help/tagsets_json.zip.
```

```
import nltk
nltk.download('maxent_ne_chunker_tab')
from nltk.chunk import ne_chunk
named_entities = ne_chunk(tagged_tokens)
print(named_entities)
```

```
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]   /root/nltk_data...
[nltk_data]   Package maxent_ne_chunker_tab is already up-to-date!
(S
  Five/CD
  Feet/NNP
  Apart/NNP
  is/VBZ
  a/DT
  romantic/JJ
  drama/NN
  directed/VBN
  by/IN
  (PERSON Justin/NNP Baldoni/NNP)
  and/CC
  released/VBN
  by/IN
  (ORGANIZATION CBS/NNP Films/NNP)
  and/CC
  (ORGANIZATION Lionsgate/NNP)
  in/IN
  2019/CD
  ./
  The/DT
  movie/NN
  stars/VBZ
  (PERSON Haley/NNP Lu/NNP Richardson/NNP)
  as/IN
```

```

(PERSON Stella/NNP Grant/NNP)
and/CC
(PERSON Cole/NNP Sprouse/NNP)
as/IN
(PERSON Will/NNP Newman/NNP)
,/,
and/CC
the/DT
story/NN
takes/VBZ
place/NN
in/IN
(GPE Saint/NNP)
Grace/NNP
Hospital/NNP
in/IN
the/DT
(GPE United/NNP States/NNPS)
./
The/DT
film/NN
raises/VBZ
awareness/NN
about/IN
(PERSON Cystic/JJ Fibrosis/NNP)
and/CC
became/VBD
popular/JJ
in/IN

```

```

for subtree in named_entities:
    if isinstance(subtree, nltk.Tree):
        entity = " ".join([word for word, tag in subtree.leaves()])
        entity_type = subtree.label()
        print(f"Entity: {entity}, Type: {entity_type}")

```

```

Entity: Justin Baldoni, Type: PERSON
Entity: CBS Films, Type: ORGANIZATION
Entity: Lionsgate, Type: ORGANIZATION
Entity: Haley Lu Richardson, Type: PERSON
Entity: Stella Grant, Type: PERSON
Entity: Cole Sprouse, Type: PERSON
Entity: Will Newman, Type: PERSON
Entity: Saint, Type: GPE
Entity: United States, Type: GPE
Entity: Cystic Fibrosis, Type: PERSON
Entity: India, Type: GPE
Entity: Canada, Type: GPE
Entity: United Kingdom, Type: ORGANIZATION
Entity: Australia, Type: GPE
Entity: Netflix, Type: PERSON
Entity: Amazon, Type: PERSON
Entity: Five Feet Apart, Type: PERSON

```

```

!pip install svgling
import svgling

```

```

Collecting svgling
  Downloading svgling-0.5.0-py3-none-any.whl.metadata (7.4 kB)
Collecting svgwrite (from svgling)
  Downloading svgwrite-1.4.3-py3-none-any.whl.metadata (8.8 kB)
Downloading svgling-0.5.0-py3-none-any.whl (31 kB)
Downloading svgwrite-1.4.3-py3-none-any.whl (67 kB)
----- 67.1/67.1 kB 5.0 MB/s eta 0:00:00
Installing collected packages: svgwrite, svgling
Successfully installed svgling-0.5.0 svgwrite-1.4.3

```

```

named_entities = ne_chunk(tagged_tokens)
named_entities

```

```

Five Feet Apart is a romantic drama directed by PERSON and released by ORGANIZATION and ORGANIZATION in
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
CD  NNP  NNP  VBZ  DT      JJ      NN      VBN      IN  Justin Baldoni CC      VBN      IN  CBS Films CC      Lionsgate IN
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP  NNP

```

```

import spacy
nlp = spacy.load ("en_core_web_sm")
text="apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)
print(doc)
for ent in doc.ents:
    print(f"Entity: {ent.text}, Label: {ent.label}")

```

```
print('Entity: {ent.text}, Label: {ent.label_}')
```

```
apple is looking at buying U.K. startup for $1 billion
Entity: apple, Label: ORG
Entity: U.K., Label: GPE
Entity: $1 billion, Label: MONEY
```

```
import nltk
from nltk import pos_tag, word_tokenize
from nltk.chunk import ne_chunk
```

```
sentence = "The quick brown fox jumps over the lazy dog"
sentence1 = "this is the class of nlp(natural language processing )"
```

```
tokens = word_tokenize(sentence1)
tagged = pos_tag(tokens)
tagged
```

```
[('this', 'DT'),
 ('is', 'VBZ'),
 ('the', 'DT'),
 ('class', 'NN'),
 ('of', 'IN'),
 ('nlp', 'NN'),
 ('(', '('),
 ('natural', 'JJ'),
 ('language', 'NN'),
 ('processing', 'NN'),
 (')', ')')]
```

```
grammar = "NP: {<DT>?<JJ>*<NN>}"
chunkParser = nltk.RegexpParser(grammar)
tree = chunkParser.parse(tagged)
```

```
chunk_parser = nltk.RegexpParser(grammar)
tree = chunk_parser.parse(tagged)
print(tree)
```

```
(S
  this/DT
  is/VBZ
  (NP the/DT class/NN)
  of/IN
  (NP nlp/NN)
  (/
    (NP natural/JJ language/NN)
    (NP processing/NN)
  ))
```

Start coding or [generate](#) with AI.